

LLMEvalIQ Field Guide

Everything this project does, why it does it, what else was on the table and why we didn't pick it — in plain language, with a real example from this exact project for every idea.

12 documents · 3,129 pages 25 gold questions 4 chunk sizes tested

3 robustness runs

CONTENTS

- 01 The one-paragraph version
- 02 Core concepts, explained simply
- 03 Phase-by-phase walkthrough
- 04 What we used vs. what else existed
- 05 Interview question bank
- 06 Quick-reference glossary
- 07 How to tell this story in an interview

SECTION 1

The one-paragraph version

We built a small system that answers questions about 12 real company annual reports by finding the relevant pages and asking an AI model to answer using only those pages — this is called **RAG**, Retrieval-Augmented Generation. Then we spent almost all our effort

not on that system, but on **proving whether it actually works**, because a RAG system that sounds confident and a RAG system that's actually right look identical from the outside.

We found the system reached the right *document* 100% of the time but found the right *passage inside it* only about 20% of the time. We ran a controlled experiment to improve that, checked the improvement survived being re-run multiple times — because AI models give different answers to the same question — analyzed exactly where and why it still fails, measured what it costs, and packaged all of it into a report, a dashboard, and a live app.

Two RAG systems, one working and one not, can look identical from the outside. This project is the instrument built to tell them apart.

SECTION 2

Core concepts, explained simply

Each term: what it means, an analogy, and a real example pulled from this exact project — not a textbook example.

RAG — Retrieval-Augmented Generation

MEANS

Instead of asking an AI to answer from memory, you first *retrieve* relevant text from your own documents, then hand that text to the AI and say "answer using only this."

ANALOGY

An open-book exam. The student doesn't need to memorize the textbook — they just need to find the right page and read it correctly.

EXAMPLE

Ask "How many board meetings did Ecoplast hold?" — the system searches thousands of chunks from 12 PDFs, pulls the 5 most relevant, and only then asks the AI to answer using just those 5 snippets.

Embedding

MEANS

A way of turning text into a list of numbers (a vector) that captures its *meaning*. Similar meaning → similar numbers, even with zero shared words.

ANALOGY

GPS coordinates for meaning — two houses close in latitude/longitude are close in space, the way two similar sentences are close in embedding space.

EXAMPLE

"What did Goa Carbon earn?" and "sales and other income was ₹70,879.78 lakhs" share almost no words, but their embeddings land close together — that's the whole trick.

Vector database

MEANS

A database built to store embeddings and quickly find the ones closest to a new one.

ANALOGY

A library shelved by topic-similarity, not title — asking for "a book about lions" surfaces nearby books on tigers too.

EXAMPLE

We used **Chroma**, storing one vector per text chunk — 22,832 of them at `chunk_size=500`.

Cosine similarity

MEANS

A -1 to 1 score for how aligned two vectors are. Chroma reports the inverse, *distance*, so smaller is better.

ANALOGY

Two arrows from the same origin — pointing nearly the same direction means "similar," regardless of length.

EXAMPLE

Genuine matches here usually score 0.55 – 0.72 . We used exactly this gap — a real chunk at 0.50 losing to a decoy at 0.65 — to diagnose the q021 failure.

Chunk, chunk size, overlap

MEANS	A document is cut into pieces ("chunks") small enough to search. Overlap is shared text between consecutive chunks, so a sentence on the cut line isn't sliced in half.
ANALOGY	Cutting a roll of fabric into curtain-sized pieces, leaving extra at each seam so no pattern is severed.
EXAMPLE	At <code>chunk_size = 500/overlap = 50</code> , a sentence could split across two chunks; at <code>600</code> there's more room for it to survive intact.

Top-K retrieval

MEANS	Only the K closest chunks (we mostly used $K=5$) are handed to the AI — not the whole database.
ANALOGY	A librarian hands you the 5 most relevant books, not the whole library.
EXAMPLE	<code>TOP_K = 5</code> across almost the whole project; Phase 5 also measured $K = 1/3/10$ to see how numbers shifted with depth.

LLM – the generation model

MEANS	The AI model that reads the retrieved chunks and the question, then writes the answer.
ANALOGY	The "student" in the open-book exam — reads what's put in front of it and answers.
EXAMPLE	Every generated answer in this project used <code>gpt-4o-mini</code> .

System prompt

MEANS

Fixed instructions given before every question, shaping behaviour no matter what's asked.

ANALOGY

The exam rules at the top of the paper: use only the material provided; write "I don't know" instead of guessing.

EXAMPLE

Ours forbids outside knowledge and mandates the exact phrase `"I don't know."` — which is why the live app says that a lot.

Token

MEANS

The real unit AI models process text in — roughly $\frac{3}{4}$ of an English word. Cost and context limits are both measured in tokens.

ANALOGY

A taxi meter billing by its own "unit," not by the kilometre you'd naturally measure in.

EXAMPLE

A typical answer used ~850–1,000 input tokens, costing about \$0.00014–0.00016.

Hallucination

MEANS

The model stating something confidently that isn't true or supported.

ANALOGY

A student inventing a plausible fact rather than leaving an exam answer blank.

EXAMPLE

Asked Infosys's operating margin, the model said 25.3% — the true figure was 20.3% — stated with total confidence.

Correctness vs. Faithfulness vs. Relevance

MEANS

Correctness: matches the truth. **Faithfulness:** every claim is backed by what was retrieved (regardless of whether that was the right thing). **Relevance:** addresses the actual question.

ANALOGY

A forecaster confidently reading yesterday's weather report for today: relevant, faithfully read, but not correct.

EXAMPLE

The Infosys margin case: faithful (the number was really on a retrieved page) + relevant (on-topic) + not correct (wrong page won retrieval) — all three at once.

Abstention

MEANS

Declining to answer rather than guessing.

ANALOGY

A witness saying "I don't recall" instead of inventing testimony.

EXAMPLE

The system abstained on roughly 52–64% of the 25 test questions — the honest alternative to far more hallucinations.

Gold set / Ground truth

MEANS

Hand-verified questions with known-correct answers and known source locations, used to actually measure the system.

ANALOGY

An answer key — without it you can only guess whether a result "sounds right."

EXAMPLE

25 questions, each with 1+ pieces of required evidence, hand-checked against the real PDF pages.

Evidence group & evidence span

MEANS

A **group** bundles alternative valid locations of the same fact (finding either counts). A **span** is that fact's exact character position in the extracted text.

ANALOGY

A treasure hunt with the same treasure buried in two marked spots — finding either wins.

EXAMPLE

q019's EPS figure genuinely appears in *two* different Infosys tables — grouped, so finding either is full credit.

LLM-as-judge

MEANS

A second AI call grades correctness/faithfulness/relevance, instead of brittle exact-string matching.

ANALOGY

A teaching assistant reading an essay for whether it answers the question — not a script checking exact words.

EXAMPLE

A separate `gpt-4o-mini` call judged each answer 3× (majority vote), then was checked against a human's own review — 100% / 92% / 100% agreement on correctness / faithfulness / relevance.

Failure taxonomy

MEANS

Fixed categories every answer sorts into, based on whether the right evidence was retrieved AND whether the answer was correct.

ANALOGY

A doctor diagnosing WHICH illness, not just writing "patient is sick" — because the fix differs by diagnosis.

EXAMPLE

Six categories used throughout: `grounded_correct`, `generation_failure`, `generation_refusal`, `unsupported_correct`, `retrieval_failure_honest`, `retrieval_failure_hallucinated`.

Pre-registered selection rule

MEANS

Deciding what counts as "success" *before* seeing the results, so the winner can't be unconsciously cherry-picked afterward.

ANALOGY

Writing down "fastest finishing time wins" before the race starts — not explaining afterward why your favourite horse "really" won.

EXAMPLE

Chunk sizes 300/500/600/1000 were compared, with the winner declared in advance to be whichever scored highest on Gold Span Coverage@5 — not Document Recall, which was maxed out everywhere.

Statistical significance — and why we didn't claim it

MEANS

A formal test for whether a difference is "real" or likely chance. We deliberately never ran one.

ANALOGY

Two heads out of three coin flips doesn't prove a biased coin — you'd need far more flips.

EXAMPLE

3 runs gave non-overlapping ranges (500: 8–12% correct; 600: 16–24%) — reported as descriptive, explicitly not as statistical proof.

Map-reduce (for summarization)

MEANS

Split a too-big document, summarize each piece ("map"), then combine those into one final summary ("reduce").

ANALOGY

Ten people each summarize 50 pages of a 500-page report; one person combines the ten summaries.

EXAMPLE

Built into the live app: documents under ~60,000 characters get one call; bigger ones map-reduce automatically.

SECTION 3

Phase-by-phase walkthrough

Phase 0

Concept

Defined the project as an **evaluation** project, not a chatbot project. The central idea, repeated throughout: retrieval and generation fail independently, so they must be

measured independently.

Phase 1

Environment setup

Python 3.12+, a curated 10-package `requirements.txt` with a WHY comment on each, a `.env` / `.env.example` split so secrets never get committed, and a smoke-test suite checking Python version, imports, and API key presence.

Phase 2

Corpus

12 real annual reports — Alivus, Alu Wind, Ecoplast, Ganesh, Goa Carbon, Infosys, Mahindra, Reliance, Rishiroop, Sayaji, Sumit, Tata Motors — **3,129 pages** total, verified readable, verified no single document dominated.

Phase 3

Gold evaluation set

25 hand-written questions across easy/medium/difficult and 5 question types, each with verified evidence. This went through real corrections — a switch from one snippet per question to a proper one-to-many schema after q010 turned out to need 4 separate facts, and manual verification of every row against the real PDF.

Phase 4

Basic RAG pipeline

Built ingestion (PDF → chunks → embeddings → vector store), retrieval (question → top-K chunks), and generation (chunks + question → answer). Provenance — source document, page numbers, character offsets — is carried through every step; this is what makes every later metric possible.

Phase 5

Frozen retrieval baseline

The measurement that set the whole project's direction:

K	DOCUMENT RECALL	ANY EVIDENCE RECALL	EVIDENCE COVERAGE
5	100.0%	24.0%	19.0%

Document Recall — did we reach the right 200-page report — saturates almost instantly. **Evidence Coverage** — did we find the actual paragraph with the answer — is

the honest number, and it's roughly 1-in-5. This gap is the single most important finding of the whole project.

Phase 6

Generation baseline

With retrieval this weak, correctness was low (8% single-run) and abstention was high (52%) — expected, not a bug. Found faithfulness $\neq$ correctness (the Infosys margin example). Found the generator gives **different wording on identical re-runs** — the reason Phase 7B exists at all.

—

Judge validation

Checked the LLM judge against a human's own review of the same 25 answers before trusting it: 100% correctness agreement, 92% faithfulness, 100% relevance. Labeled explicitly as a smoke test, and the human labels as **AI-assisted manual review**, not independent blind annotation — because that's what they actually are.

Phase 7A

Chunk-size experiment

Tested `chunk_size` $\in$ {300, 500, 600, 1000}, overlap fixed at 50. Winner selected by a rule written down *before* running generation.

CHUNK_SIZE	GOLD SPAN COVERAGE@5	CHUNKS	INDEX SIZE	BUILD TIME
500 — baseline	19.7%	22,832	259.1 MB	552.9s
600 — winner	23.9%	18,682	222.9 MB	491.7s

Document Recall was deliberately *not* the deciding metric — it read 100% for every chunk size, so it had zero power to distinguish a good choice from a bad one.

Regression found, not hidden: q021 (Infosys client numbers) got worse at `chunk_size=600`. Direct investigation found the cause: a neighbouring client-count TABLE scored higher similarity (0.65) than the chunk with the real answer (0.50, below the usual "real match" floor) — the retriever picked the wrong-but-topically-similar content.

Phase 7B

End-to-end robustness — 3 independent runs

Because generation is non-deterministic, the FULL pipeline was run 3 separate times per configuration:

	500 – BASELINE	600 – WINNER
Correctness	mean 10.7%, range 8–12%	mean 20.0%, range 16–24%
Faithfulness	mean 84.0%	mean 92.0%

The ranges don't overlap — reported descriptively, explicitly not a formal significance claim with only 3 runs. Also surfaced `unsupported_correct`: 5 of 75 answers at 600 were correct with **no retrieved evidence supporting them** — most plausibly the model's own training knowledge, not the pipeline. Never counted as a win.

Phase 8

Failure analysis

Sliced the taxonomy by document, difficulty, and question type to find WHERE failures cluster. **Goa Carbon** abstained honestly 100% of the time (its chunks embed unusually poorly against natural questions). **Infosys** hallucinated 3.6× more than average (its tables read as topically confident even when wrong). **Ecoplast** concentrated on the generation side. Categories with too few questions ($n \leq 4$) were flagged as unreliable, not treated as findings.

Phase 9

Cost, latency, efficiency

Verified `gpt-4o-mini` pricing fresh against OpenAI's page before computing any dollar figure. Found three separate, sometimes-contradicting truths:

- **Retrieval infrastructure:** 600 is better on every axis — 18.2% fewer chunks, 14.0% smaller index, 11.1% faster build.
- **Generation token cost:** 500 is cheaper per call — 600 uses ~18% more input tokens per question.
- **Outcome efficiency:** 600 is cheaper *per correct answer* — \$0.00080 vs \$0.00129.

Phase 10

Final report, dashboard, README, audit

One coherent report instead of pasted phase outputs, an **offline** dashboard that never calls an API, a portfolio README, and a repository audit. One real finding fixed:

`.gitignore` was silently excluding the entire `results/` folder — every number this project produced would have been invisible to anyone cloning the repo.

Bonus **Live Q&A app**

Built after Phase 10, kept deliberately separate from the offline dashboard. Three modes — **Ask a question** (top-K retrieval), **Summary**, and **Notes** (both read the whole document, since summarizing needs everything, not a 5-chunk slice). Supports uploading your own PDF into its own separate index, so it can never overwrite the curated corpus.

SECTION 4

What we used vs. what else existed — and why

DECISION	WHAT WE USED	REAL ALTERNATIVES	WHY WE PICKED
Embedding model	<code>text-embedding-3-small</code>	<code>text-embedding-3-large</code> ; open-source <code>sentence-transformers</code> ; Cohere Embed	Cheap enough to run repeatedly across many excluded test cases
Vector database	Chroma (local, on disk)	Pinecone / Weaviate / Qdrant (managed, cloud-scale); FAISS (fast library, no metadata storage)	Zero external dependencies; built-in metadata storage; provenance tracking
Generation model	<code>gpt-4o-mini</code>	<code>gpt-4o</code> (~15× pricier); Claude; open-source Llama/Mistral	Cheap enough to run many times across many test cases; v1 excluded
Chunking strategy	Fixed-size overlapping character chunks	Semantic chunking; sentence-based; table-aware chunking	Simple, deterministic; experimentally chosen; Table-aware chunking was a lot of work, not built-in
Judging method	LLM-as-judge, checked against human labels	Exact string match; BLEU/ROUGE word-overlap metrics; human-only grading	String match and BLEU/ROUGE scored correctly on different numbers

Evidence matching	Fixed character-offset spans + groups of alternatives	Simple per-chunk text search; single fixed answer location	The ruler has t compared, or suits the ruler.
Chunk-size winner rule	Gold Span Coverage@5, decided before results	Document Recall (saturated, no signal); eyeballing after seeing generation numbers	Pre-registering finding trustw after the fact.
Robustness check	3 independent full runs, ranges reported descriptively	1 run; a formal p-value significance test	3 runs is the n without preter can't support.
Dashboard	Streamlit	Flask + custom React; Gradio; a static notebook export	Turns a Python almost no extr testing.
Deliberately not built (v1)	—	Multiple LLMs/embeddings; agent evaluation; prompt-injection testing; CI/CD; a human-eval platform; advanced statistics; Docker	Each is legitim would have ch work, and by h

SECTION 5

Interview question bank

Q Walk me through this project in two minutes.

I built a RAG system over 12 real annual reports, then spent most of my effort proving whether it actually worked rather than assuming it did. The first real measurement found the system reached the right document 100% of the time but found the answer-bearing passage only ~20% of the time. I ran a controlled, pre-registered experiment across four chunk sizes and found one that improved evidence coverage by about 21% relative. Because generation is non-deterministic, I re-ran the full pipeline three times per configuration before trusting that improvement — it held, correctness roughly doubled. Then I did failure analysis to find where it still breaks, and a cost analysis to understand what

the improvement costs. Everything's packaged into a report, an offline dashboard, and a live app.

Q Why isn't "the system found the right document" a good enough metric?

On a 200+ page report, almost any related question retrieves a chunk from the right document — that's an easy bar. It says nothing about whether the specific paragraph needed was found. Document Recall was 100% while Evidence Coverage was only ~20%. Reporting only the first number makes a mostly-broken system look like it works.

Q How do you know your LLM judge is trustworthy?

I checked its verdicts against a human's own review of the same answers before trusting any of its numbers — 100% correctness agreement, 92% faithfulness, 100% relevance. I'm explicit that this is a small-sample smoke test, not a calibrated instrument, and that the "human" labels were AI-assisted, not independent blind annotation.

Q Difference between correctness and faithfulness — why do you need both?

Correctness asks "does this match the truth?" Faithfulness asks "is every claim backed by what the model was shown?" My clearest example: asked for Infosys's operating margin, the model confidently answered 25.3% when the truth was 20.3% — faithful (it really read that number off a retrieved page) and relevant, but not correct. Faithfulness alone told me it wasn't hallucinating from nowhere; it was reading a genuinely wrong source.

Q How did you pick the winning chunk size, and how do you know it wasn't cherry-picked?

I wrote the selection rule down — Gold Span Coverage@5 — before running any generation, and deliberately avoided Document Recall since it was saturated everywhere. Writing the rule first is what makes the result defensible.

Q You said correctness "roughly doubled." Is that statistically significant?

No, and I never claim it is. Three runs gave non-overlapping ranges (8–12% vs 16–24%), a real descriptive signal — but three data points can't support a formal significance claim, and I say that explicitly.

Q **What is `unsupported_correct` and why does it matter?**

An answer that matches the truth with NO retrieved evidence supporting it — most plausibly the model already knew the fact from training, not from my retrieval. I never count these as pipeline wins; they're a contamination risk, and hiding them would flatter the system.

Q **Tell me about a specific failure you root-caused.**

q021 asked about Infosys's new client count. At the winning chunk size, the correct chunk was intact and indexed — but a neighbouring client-count table scored a higher similarity (0.65 vs 0.50) and got retrieved instead. I confirmed this by directly embedding both chunks and comparing scores. It's an embedding-ranking failure, not a completeness failure — the right text existed, it just lost a popularity contest to a wrong-but-similar table.

Q **How do you decide what NOT to build?**

I followed an explicit v1 scope excluding multi-LLM testing, agent evaluation, CI/CD, advanced statistics, Docker, and more. Each is legitimate for a bigger version, but none would have changed the answer to this project's actual question.

Q **Is a "better" configuration always more expensive?**

Not necessarily — separating those questions was one of the more important findings. The winning chunk size costs ~18% more PER CALL (bigger chunks, more prompt text) but wastes far fewer calls on wrong answers, so it's CHEAPER per correct answer — \$0.00080 vs \$0.00129. "Cheaper per call" and "cheaper per successful outcome" are genuinely different claims.

Quick-reference glossary

RAG

Retrieve relevant text first, then generate an answer using only that text.

Embedding

Text turned into numbers that capture meaning.

Chunk

A piece of a document, small enough to search and hand to an AI model.

Top-K

Only the K most relevant chunks are used, not the whole document.

Cosine similarity

How aligned two embeddings are — closer to 1 means more similar meaning.

Hallucination

A confident but false or unsupported claim.

Correctness

Does the answer match the truth?

Faithfulness

Is the answer backed by what was actually retrieved?

Relevance

Does the answer address the question asked?

Abstention

Saying "I don't know" instead of guessing.

Gold set

Hand-verified questions and correct answers, used to measure the system.

Evidence span

The exact character location of a fact in the source document.

LLM-as-judge

Using a second AI call to grade an answer, checked against human judgment.

Failure taxonomy

Fixed categories every answer sorts into, based on why it failed.

Pre-registered rule

Deciding the success metric before seeing the results.

Document Recall

Did we reach the right document? Easy, often saturated.

Evidence Coverage

Did we find the actual answer-bearing passage? The honest number.

Map-reduce

Summarize big documents in pieces, then combine the pieces.

SECTION 7

How to tell this story in an interview

1. **The hook** — two RAG systems can look identical from the outside; this project is the instrument built to tell them apart.
2. **The headline finding** — 100% Document Recall, ~20% Evidence Coverage. The whole project's reason for existing.
3. **The experiment** — a pre-registered chunk-size test, one metric decided in advance, an honest regression (q021) reported alongside the win.
4. **The robustness check** — why one run isn't enough, and exactly what three runs did and didn't prove.
5. **The cost nuance** — "better" isn't one number; infrastructure, generation cost, and outcome efficiency can point different directions at once.
6. **The discipline** — what you refused to claim (significance, independent human validation) matters as much as what you found.

"Document recall was already 100% while evidence coverage was only ~20% — and a controlled, pre-registered experiment closed part of that gap while nearly doubling end-to-end correctness, without ever claiming more statistical certainty than three runs can actually support."

